



Practical 1 Memo: The Study Squad Tracker

COS 132 — Imperative Programming

Practical Memo

Programme note. The premium COS 132 course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. This material is Day 1 of the 7-day programme.

Good day, good people. This memo walks you through every function in the practical from first principles. The point is not just to show you the answer; the point is to make sure you can write the same kind of code from scratch the next time you see a problem like it.

Read this **after** you have made an honest attempt. If you read it before, you skip the bit where the concept actually sticks.

How this memo is organised

For each function:

- **What it teaches** – the concept the function is exercising
- **How to think about it** – a plain-English approach you can apply yourself
- **The implementation** – the actual code
- **Line by line** – what each piece of code is doing and why
- **Common pitfalls** – the mistakes that catch students out

At the end there is a section on **how the test harness works** so you understand what `main.cpp` is doing and why each test feeds the kind of input it does.

Setting the scene: the namespace and the variables

Before any of the functions, the file `Squad.h` declares a namespace called `SQUAD` and four extern variables. The starter `Squad.cpp` defines those variables with their initial values. You should not need to change anything in this section, but it is worth understanding what is happening.

```
// Inside namespace SQUAD, in Squad.h:  
extern std::string memberName;  
extern int totalHours;  
extern int sessionsAttended;  
extern double quizAverage;
```

`extern` says "this variable exists somewhere, just not in this file." The actual definition (with the initial value) lives in `Squad.cpp`:

```
namespace SQUAD {  
    std::string memberName = "";  
    int totalHours = 0;  
    int sessionsAttended = 0;  
    double quizAverage = 0.0;  
}
```

This is the standard pattern for sharing variables across multiple files in C++. Each file that uses these variables only needs to `#include "Squad.h"` and the names become available, but there is only one copy of each variable in memory (the one in `Squad.cpp`).

In Practical 1 you do not actually use these variables yourself, but the pattern is here because the practical is mirroring the structure you will see in real COS 132 practicals.

Group 1: Arithmetic Functions

addHours

What it teaches: the simplest possible accumulator pattern. Add two numbers and return the result.

How to think about it: a "running total" keeps track of how much you have so far. Every time something new arrives, you add it to the running total. The function takes the current total and the new amount, returns the sum.

Implementation:

```
int addHours(int currentTotal, int newHours) {  
    return currentTotal + newHours;  
}
```

Line by line:

- `int addHours(...)` – the function returns an `int`. The two parameters are also `ints`.
- `return currentTotal + newHours;` – compute the sum and send it back. That is the entire function.

Common pitfalls:

- Forgetting to return at all (just writing `currentTotal + newHours;` as a statement). The compiler will warn you about this.
- Trying to modify a global variable instead of returning the result. The function should be pure: it takes inputs, returns an output, no side effects.

incrementSessions

What it teaches: a counter that goes up by 1.

How to think about it: if a member has attended 3 sessions and shows up to one more, they have now attended 4. That is what "increment" means: add one.

Implementation:

```
int incrementSessions(int currentCount) {  
    return currentCount + 1;  
}
```

Line by line:

- Take the current count, add one, return it. That is the whole job.

Common pitfalls:

- Writing `return currentCount++;` instead of `return currentCount + 1;`. The first one returns the **old** value before incrementing. Subtle and wrong.
- Writing `return ++currentCount;` is functionally correct (returns the new value), but it also modifies the parameter, which is unnecessary. Stick with `currentCount + 1`.

calculateAverage

What it teaches: integer-versus-double division and the divide-by-zero edge case.

How to think about it: an average is a total divided by a count. But there are two traps. The first trap is integer division: if you divide an `int` by an `int`, you lose the fractional part. The second trap is dividing by zero. With **integer** division by zero, the C++ standard says the result is undefined and your program may crash. With **floating-point** division by zero (which is what we have here, because `total` is a `double`), the IEEE-754 standard says the result is `inf` or `nan` instead. Either way the answer is meaningless, so we guard against it before doing the division at all.

Implementation:

```
double calculateAverage(double total, int count) {
    if (count == 0) {
        return 0.0;
    }
    return total / count;
}
```

Line by line:

- `if (count == 0) return 0.0;` – this is the divide-by-zero guard. If there are no items, an “average” is meaningless, so we just return 0.0 by convention. The spec tells you to do this exactly.
- `return total / count;` – the actual division. Notice that `total` is a `double` and `count` is an `int`. When you divide a `double` by an `int`, C++ promotes the `int` to a `double`, and the division happens in floating-point. So `75.0 / 3` correctly gives `25.0`, not 25 truncated from anything.

Why the guard order matters: The guard sits **before** the division. If you flipped it (do the division first, then check), the division would have already happened and produced `inf` or `nan`, and your function would print garbage. Always check for divide-by-zero **before** you divide.

Common pitfalls:

- Forgetting the divide-by-zero check entirely. Test 4 specifically passes `count = 0` and expects 0. Without the guard, your program will produce `inf` or `nan`, and Test 4 fails.
- Writing `if (count == 0.0)` – syntactically fine because of implicit conversion, but it is comparing an `int` to a floating-point literal. Cleaner to write `if (count == 0)`.
- Performing integer division by writing `int avg = total / count;`. This loses the fractional part. The function returns `double` for a reason.

scaleHours

What it teaches: mixed-type arithmetic. What happens when an `int` and a `double` get multiplied?

How to think about it: you have a number of hours (an integer) and you want to scale it by some factor (a fractional value, like 1.5 for “exam-week boost”). The result might not be a whole number, so it has to be a `double`.

Implementation:

```
double scaleHours(int hours, double factor) {
    return hours * factor;
}
```

Line by line:

- `hours * factor` – C++ sees an `int` being multiplied by a `double`. The rule: when the two operands have different numeric types, the smaller one is promoted to the larger one. `int` gets promoted to `double`, the multiplication happens in floating-point, and the result is a `double`.

Common pitfalls:

- Returning `int` accidentally (if you change the function signature). The return type must be `double` to preserve the fractional result.
- Trying to cast manually: `static_cast<double>(hours) * factor`. This is also correct, but unnecessary. The implicit promotion handles it for you. Cleaner code is better code.

Group 2: Boolean Functions

`isOnFire`

What it teaches: a single-condition boolean function with a "greater than or equal to" comparison.

How to think about it: a member is "on fire" if they have shown up enough. The threshold here is 5 sessions. So the question reduces to: is `sessionsAttended` greater than or equal to 5? Whatever that comparison evaluates to is the answer.

Implementation:

```
bool isOnFire(int sessionsAttended) {  
    return sessionsAttended >= 5;  
}
```

Line by line:

- `sessionsAttended >= 5` is itself a boolean expression. It evaluates to either `true` or `false`. So you can return it directly, no `if/else` needed.

Common pitfalls:

- Writing it as an `if/else` block:

```
if (sessionsAttended >= 5) {  
    return true;  
} else {  
    return false;  
}
```

This works but it is verbose. The single-line return is cleaner and produces identical code. Prefer it.

- Using `>` instead of `>=`. The spec says "5 or more", which includes 5. `>` would exclude 5.

`isPassing`

What it teaches: the same shape as `isOnFire`, but with a `double` comparison.

How to think about it: a quiz average of 50 or above is a pass. Same idea, different threshold and different type.

Implementation:

```
bool isPassing(double quizAverage) {  
    return quizAverage >= 50.0;  
}
```

Line by line:

- `quizAverage >= 50.0` – compare the double value against the threshold and return the result.

Common pitfalls:

- Comparing against 50 (an int literal) instead of 50.0. C++ will promote 50 to 50.0 for you, so it works, but writing 50.0 makes the type-matching obvious.
-

canTutor

What it teaches: combining two conditions with `&&` (logical AND).

How to think about it: a member can tutor only if **both** of these are true:

1. Their average is at least 75
2. They have attended at least 8 sessions

If either one fails, they cannot tutor. So you join the two conditions with `&&`. `&&` returns true only if both sides are true.

Implementation:

```
bool canTutor(double quizAverage, int sessionsAttended) {  
    return quizAverage >= 75.0 && sessionsAttended >= 8;  
}
```

Line by line:

- The `&&` is short-circuit. If `quizAverage >= 75.0` is false, C++ does not even evaluate the second half (it knows the answer is already false). That is fine here; both halves are pure comparisons with no side effects, but the principle is good to know.

Why this is Caleb's function: Caleb keeps asking when he qualifies to tutor. The thresholds are deliberately set so that he has to show up consistently AND maintain a strong average. Either alone is not enough.

Common pitfalls:

- Using `||` (OR) instead of `&&`. With OR, getting either condition true would qualify you, which is wrong: a person with 5 sessions and a 90% average is not yet ready to tutor (they have not shown up enough yet).
 - Wrapping each comparison with `()` excessively: `((quizAverage >= 75.0) && (sessionsAttended >= 8))`. The extra parentheses are not wrong, but they are noise. C++ comparison operators bind tighter than `&&`, so the original form parses the same way.
-

Group 3: Overloaded Function

setReminder(int hours) – the simple version

What it teaches: producing exact-format output with cout.

How to think about it: the function takes a number of hours and prints a single line showing that the reminder is set. No conversion, no logic. Just print.

Implementation:

```
void setReminder(int hours) {  
    std::cout << "Reminder set for " << hours << "h" << std::endl;  
}
```

Line by line:

- `std::cout` – the standard output stream.
- `<< "Reminder set for "` – print the literal string, including the trailing space.
- `<< hours` – print the integer value.
- `<< "h"` – print the literal h.
- `<< std::endl` – print a newline and flush the buffer.

Common pitfalls:

- Forgetting the space after for (writing "Reminder set for" without the trailing space). The output becomes `Reminder set for3h` – no space between for and 3. Test 10 fails.
- Forgetting the h after the number.
- Using `"\\{\\}n"` instead of `endl`. Both work and produce identical output for grading purposes. Either is fine.

setReminder(int value, std::string unit) – the overloaded version

What it teaches: function overloading and a small piece of conversion logic.

How to think about it: this version takes a value and a unit. If the unit is "minutes", you need to convert to hours by dividing by 60. If the unit is anything else (specifically "hours" per the spec), you use the value as-is. After the conversion, the print is exactly the same as the simple version.

The overloading part: there are now two functions both called `setReminder`. C++ picks which one to use based on what you pass. `setReminder(3)` calls the one-parameter version. `setReminder(120, "minutes")` calls the two-parameter version. The compiler decides at compile time, based on the arguments.

Implementation:

```
void setReminder(int value, std::string unit) {  
    int hours = value;  
    if (unit == "minutes") {  
        hours = value / 60;  
    }  
    std::cout << "Reminder set for " << hours << "h" << std::endl;  
}
```

Line by line:

- `int hours = value;` – start by assuming the value is already in hours. This is the default case ("hours" unit).
- `if (unit == "minutes") hours = value / 60;` – if the unit is "minutes", convert. Note this is **integer** division (both `value` and `60` are `int`), so `120 / 60 = 2` exactly, but `90 / 60 = 1` (the 0.5 hour is truncated). The spec explicitly says integer division, so this is intended.
- The print statement is the same as the simple version.

Common pitfalls:

- Comparing strings with `==`. This actually works for `std::string` because the class has an `==` operator that compares character-by-character. (For C-strings – `char*` – you would need `strcmp`, but here we have proper `std::string`.)
- Doing `value / 60.0` (floating-point division). The spec says integer division. `value / 60.0` would give 1.5 for 90 minutes, which is wrong per the spec.
- Forgetting to handle the "hours" case at all. If `unit` is "hours" you still need to print – the easiest way is to start with `int hours = value` and only override it inside the `if`. Then the print statement at the bottom always runs.
- Having two separate `cout` blocks (one in the `if`, another in the `else`). It works, but it duplicates the print code. Cleaner to compute hours first and print once.

Group 4: Report Function

printSessionLog

What it teaches: producing strict, multi-line formatted output. The formatting must match exactly.

How to think about it: Yolisa wants this report to look right. That means every dash, every space, every newline matters. The way to write this is one `cout` per line, each ending in `endl` (or `"\\{\\}n"`).

Implementation:

```
void printSessionLog(std::string name, int sessions, int hours, double average) {
    std::cout << "-----SQUAD MEMBER LOG-----" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Sessions: " << sessions << std::endl;
    std::cout << "Hours: " << hours << std::endl;
    std::cout << "Average: " << average << std::endl;
    std::cout << "-----" << std::endl;
}
```

Line by line:

- Each line of output is a separate `cout` statement. This is easier to read and easier to debug than trying to chain everything together.
- The opening dashes (`---SQUAD MEMBER LOG---`) are 5 dashes, the title, then 5 more dashes. Count them.
- The closing line is 26 dashes – exactly enough to underline the longest content. You can count them in the spec: the line is `-----` (26 characters).
- Each label has a colon and a single space: `Name:` , `Sessions:` , etc. The space between the colon and the value is important.

Common pitfalls:

- Wrong number of dashes. Count carefully: 5 dashes, "SQUAD MEMBER LOG", 5 dashes for the opening. 26 dashes for the closing.
- Missing the space after the colon (Name:Yolisa instead of Name: Yolisa).
- Adding an extra space (Name: Yolisa).
- Forgetting the newline at the end of any line.
- Adding an extra newline at the end. The grader compares output exactly: too many newlines and the test fails.

Yolisa's diagnostic test: if your output is one character off, your test fails. So if you got 8/10 marks but cannot see why, copy your output into a text editor and put it next to the spec output. Look character by character. Yolisa would.

Group 5: Orchestrator

processMember

What it teaches: putting everything together. The orchestrator function calls the smaller functions you have already written and uses their return values.

How to think about it: by the time you get here, you have already built all the pieces. processMember is just deciding the order and adding a few cout lines for the booleans.

The order is fixed:

1. Print the session log first (so the reader sees who and what is being processed).
2. Compute the three boolean checks.
3. Print each result on its own line.
4. Return whether all three are true.

Implementation:

```
bool processMember(std::string name, int sessions, int hours, double average) {
    printSessionLog(name, sessions, hours, average);
    bool fire = isOnFire(sessions);
    bool pass = isPassing(average);
    bool tutor = canTutor(average, sessions);
    std::cout << "On fire: " << fire << std::endl;
    std::cout << "Passing: " << pass << std::endl;
    std::cout << "Can tutor: " << tutor << std::endl;
    return fire && pass && tutor;
}
```

Line by line:

- printSessionLog(...) – call your own function. Because you are inside the SQUAD namespace, you do not need to qualify it as SQUAD::printSessionLog.
- The three boolean computations – store them in local variables so you can both print them AND combine them at the end without computing anything twice.
- The three cout statements print each boolean. C++ prints true as 1 and false as 0 by default. That matches the spec.
- return fire && pass && tutor; – only true if all three are true. This is the same && chain you used in canTutor, just with three operands.

Common pitfalls:

- Calling `printSessionLog` after the boolean printing (wrong order – the spec requires the log first).
 - Re-computing the booleans inside the return statement: `return isOnFire(sessions) && isPassing(average) && canTutor(...)`; This works but it computes each function twice (once to print, once to return). Slightly wasteful; not a correctness issue but a code quality one.
 - Forgetting that the test harness prints `Squad Star: <result>` after `processMember` returns. That line is in `main.cpp`, not in your function. So `processMember` itself does not print "Squad Star".
 - Mixing up the parameter order when calling `canTutor`. The spec for `canTutor` is `canTutor(quizAverage, sessionsAttended)` – average first, sessions second. Inside `processMember` the parameters are also in that order, so `canTutor(average, sessions)`.
-

How the test harness works

Open `main.cpp` and read it. It is a simple dispatcher:

1. Read an integer `test_id` from standard input.
2. Based on the value of `test_id`, read whatever extra inputs that test needs.
3. Call the relevant `SQUAD::` function and print the result.

So when you run `./squad < tests/inputs/01.txt`, the program reads:

```
1
10 5
```

The first number is the test ID (1). The harness sees test ID 1, knows that test 1 is for `addHours`, and reads two more integers (10 and 5). Then it calls `SQUAD::addHours(10, 5)` and prints the result.

The expected output for test 1 is in `tests/expected/01.txt`:

```
15
```

`grade.sh` reads the expected file, runs your binary on the input file, and compares the two strings. If they match exactly (after stripping any carriage-return characters so the grader works across Linux and Windows shells), the test passes.

Why the test driver design is useful: every test is isolated. If you are debugging Test 7, you can run only that test with:

```
./squad < tests/inputs/07.txt
```

You do not have to run the whole battery to check one function.

A reminder about the running joke

The variables, the function names, the prac scenario – they are all named after the four people who actually use this material every week. Caleb really does call Swift "Uncle". Yolisa really does notice when output formatting is one space off. Sahana really does keep an attendance log in a notebook. Swift really does write the code that ties everyone together.

Take it seriously, but not too seriously.

When you finish this prac, run `./grade.sh` one last time, screenshot the output, and post it in the group chat. Caleb will probably reply with a comment about how he wrote his version faster. Ignore him. He is your sibling-in-coding now.

Get to it.